

Sistemas de Consenso e o Problema dos Generais Bizantinos.

Kenyo Boechat¹, Arthur Borges¹, Robert Silva¹, Ester de Lima¹

¹Sistemas de Informação – Universidade Federal do Mato Grosso do Sul(UFMS)

{kenyo.boechat, arthur_b, robert.jonathan, ester.lima}@ufms.br

Abstract. *This meta-paper describes the style to be used in articles and short papers for SBC conferences. For papers in English, you should add just an abstract while for the papers in Portuguese, we also ask for an abstract in Portuguese (“resumo”). In both cases, abstracts should not have more than 10 lines and must be in the first page of the paper.*

Resumo. *Este meta-artigo descreve o estilo a ser usado na confecção de artigos e resumos de artigos para publicação nos anais das conferências organizadas pela SBC. É solicitada a escrita de resumo e abstract apenas para os artigos escritos em português. Artigos em inglês deverão apresentar apenas abstract. Nos dois casos, o autor deve tomar cuidado para que o resumo (e o abstract) não ultrapassem 10 linhas cada, sendo que ambos devem estar na primeira página do artigo.*

1. O que é Consenso Distribuído

Em sistemas de computação distribuída, o consenso é o processo fundamental pelo qual múltiplos participantes independentes (nós) chegam a um acordo coletivo sobre um único valor ou estado. Este mecanismo é a pedra angular para manter a consistência e a confiabilidade em qualquer sistema descentralizado, desde bancos de dados replicados em nuvem até as redes de blockchain que alimentam criptomoedas. O objetivo é garantir que todos os processos corretos e honestos no sistema concordem sobre o estado atual dos dados, como quais transações foram validadas e em que ordem.

A necessidade de algoritmos de consenso complexos surge diretamente da inevitabilidade das falhas em redes distribuídas. Se todos os nós e canais de comunicação fossem perfeitamente confiáveis, chegar a um acordo seria trivial. No entanto, em sistemas do mundo real, os componentes falham. A complexidade e a robustez de um algoritmo de consenso são, portanto, definidas pelo tipo de falha que ele é projetado para tolerar.

1.1. Falha de Parada (Crash Fault)

O modelo de falha mais simples e benigno é a “falha de parada” (ou crash fault). Neste modelo, um nó que falha é considerado passivo. Ele é caracterizado por uma parada prematura e abrupta. O nó simplesmente não responde e não executa novas operações. Este tipo de falha é o mais fácil de detectar; para o resto da rede, o nó fica em silêncio. Crucialmente, o modelo de falha de parada assume que os nós são honestos, mas frágeis: eles podem “morrer”, mas nunca mentirão ou agirão maliciosamente.

1.2. A Falha Bizantina (Byzantine Fault)

Em contraste nítido, a "falha bizantina" representa o modelo de falha mais severo e complexo. Um nó que sofre uma falha bizantina exibe comportamento arbitrário. Este nó não está apenas quebrado; ele é ativamente não confiável e pode ser considerado um agente malicioso.

O comportamento arbitrário de um nó bizantino inclui, mas não se limita a:

- Parar de responder: Agir como uma falha de parada.
- Corromper dados: Enviar mensagens incorretas ou respostas erradas.
- Ser seletivo: Responder a alguns nós, mas ignorar outros.
- Equivocar (Mentir Ativamente): Este é o comportamento mais destrutivo. Um nó bizantino pode responder que aprova um bloco b para um participante e que aprova um bloco b' para outro participante.

Este último ponto é o que torna as falhas bizantinas tão difíceis de resolver. O nó está ativamente tentando enganar a rede e destruir a consistência do sistema, criando múltiplas versões da verdade. É importante notar que o comportamento bizantino não é exclusivamente malicioso. Pode também ser de origem benigna, causado por bugs de software, corrupção de memória ou falhas de hardware sutis.

2. O Problema dos Generais Bizantinos (BGP)

O desafio de projetar um sistema Tolerante a Falhas Bizantinas foi brilhantemente abstraído e formalizado em 1982 por Leslie Lamport, Robert Shostak e Marshall Pease no artigo seminal "The Byzantine Generals Problem" (O Problema dos Generais Bizantinos).

2.1. A Analogia

A analogia, agora um clássico da ciência da computação, descreve um cenário militar :

1. Atores: Várias divisões de um exército bizantino, cada uma comandada por um General (representando um nó ou processo computacional), cercam uma cidade inimiga.
2. Objetivo: Os generais devem decidir coletivamente por um plano de batalha comum: "atacar" ou "recuar".
3. Comunicação: Eles só podem se comunicar por mensageiros (representando uma rede), que podem ser interceptados ou atrasados.
4. A Falha (O Traidor): Um ou mais generais podem ser "traidores" (os nós bizantinos). Um general traidor não apenas votará de forma contrária, mas agirá maliciosamente. Ele pode, por exemplo, enviar uma mensagem de "ataque" para o General A e uma mensagem de "recue" para o General B, numa tentativa de dividir as forças leais.
5. Condição de Vitória: A coordenação é tudo. Se todos os generais leais atacarem simultaneamente, eles vencem. Se alguns atacarem e outros recuarem (ou atacarem em momentos diferentes), eles serão derrotados.

O dilema central é: como um general leal pode chegar a um consenso com seus pares quando ele não pode confiar em nenhuma das mensagens que recebe?. Este problema é exclusivo de sistemas descentralizados. Um sistema centralizado o evita ao ter uma autoridade central confiável (um "Comandante-em-Chefe") cuja ordem é final e inquestionável. Em uma rede peer-to-peer , não existe tal autoridade; os generais devem criar a verdade por si mesmos.

2.2. A Formalização: Condições de Consistência Interativa

Lamport, Shostak e Pease traduziram essa analogia em duas condições formais, chamadas de Condições de Consistência Interativa (IC), que qualquer solução BFT deve satisfazer. No problema formal, um Comandante envia uma ordem para seus $n-1$ Tenentes:

- IC1. Todos os tenentes leais obedecem à mesma ordem. (Esta é a propriedade de Consistência ou Acordo).
- IC2. Se o general comandante for leal, então todo tenente leal obedece à ordem que ele envia. (Esta é a propriedade de Validade ou Correção).

A derrota na analogia não vem de tomar a decisão errada (ex: todos recuarem), mas de tomar decisões divididas (metade ataca, metade recua). O Problema dos Generais Bizantinos, portanto, prioriza a consistência (todos concordam em uma única ação) acima de tudo.

3. Teoria e limites: A Solução do BGP

Primeiro, os autores consideraram um sistema de Mensagens Orais (OM). Na analogia, uma mensagem oral é aquela em que o mensageiro entrega a ordem, mas não há como provar sua origem ou autenticidade. Um tenente traidor pode mentir sobre a ordem que recebeu do comandante (ex: O comandante me disse recue) e não há como verificar. Isso é análogo a pacotes de rede IP não autenticados.

Para este modelo, o artigo apresenta um resultado de impossibilidade surpreendente: Uma solução para o problema BGP com mensagens orais só é possível se e somente se mais de dois terços $2/3$ dos generais forem leais. Isso é formalizado como $n > 3f$, onde n é o número total de generais e f é o número de traidores. Dito de outra forma, o sistema deve ter $n \geq 3f + 1$ nós para tolerar f falhas bizantinas.

3.1. A Solução Teórica: Mensagens Assinadas (SM)

O artigo então propõe uma segunda solução que muda as premissas do sistema: Mensagens Assinadas (SM). Uma mensagem assinada é análoga a uma mensagem com uma assinatura digital criptográfica (ex: RSA) que é inforjável.

As novas premissas são :

1. A assinatura de um general leal não pode ser forjada.
2. Qualquer pessoa pode verificar a autenticidade da assinatura de um general.

Isso muda o jogo fundamentalmente. O problema com as Mensagens Orais era a negação plausível. As Mensagens Assinadas removem isso e transformam mentiras em evidências irrefutáveis de malícia.

4. Consenso Tolerante a falhas de parada (CFT)

Dado o alto custo da tolerância BFT (tanto em número de nós, $n > 3f$, quanto em complexidade computacional), a grande maioria dos sistemas distribuídos práticos optou por resolver um problema mais simples: a Tolerância a Falhas de Parada (CFT).

Dois dos algoritmos CFT mais influentes são o Paxos e o Raft:

- Paxos: Desenvolvido por Leslie Lamport, Paxos é uma "família de protocolos" considerada a base teórica do consenso CFT. É conhecido por sua robustez e correção matemática, mas também por sua notória complexidade de implementação e compreensão.
- Raft: Desenvolvido em 2013, o Raft foi explicitamente "projetado para ser mais fácil de entender e implementar que o Paxos". Ele funciona através de uma abordagem baseada em líder, onde um nó é eleito líder e é o único responsável por gerenciar o log de replicação. Se o líder falhar (crash), uma nova eleição é acionada para escolher um novo líder.

5. BFT Determinístico: Practical Byzantine Fault Tolerance (PBFT)

O PBFT foi o primeiro algoritmo a demonstrar que a Tolerância a Falhas Bizantinas (BFT) poderia ser alcançada com desempenho suficiente para sistemas do mundo real.

O artigo de Castro e Liskov tornou o BFT prático ao resolver dois grandes problemas dos algoritmos anteriores :

1. Funciona em Ambientes Assíncronos: Algoritmos anteriores muitas vezes assumiram um sistema síncrono (onde há limites máximos conhecidos para o atraso da rede). O PBFT foi projetado para funcionar em ambientes assíncronos como a Internet. Ele não depende do timing da rede para garantir a segurança (consistência), apenas para garantir a vivacidade (que o sistema continue progredindo).
2. Otimização de Desempenho: O PBFT foi mais de uma ordem de magnitude (10x) mais rápido que as soluções BFT anteriores. A otimização chave foi evitar o uso de criptografia de chave pública (assinaturas digitais) no caminho feliz (operação normal). Em vez disso, o PBFT usa Códigos de Autenticação de Mensagem (MACs) simétricos, que são muito mais rápidos. A cara criptografia de chave pública só é usada durante as mudanças de visão (protocolos de recuperação quando um líder falha)

6. BFT Probabilístico

O Consenso Nakamoto é uma solução probabilística para o BGP, em contraste fundamental com o PBFT, que é determinístico.

Em vez de um protocolo de votação complexo (como o do PBFT), o NC usa um mecanismo de consenso preguiçoso. Um general (nó) bizantino é livre para mentir (propor um bloco em uma cadeia alternativa). No entanto, para que essa mentira seja aceita pelo resto da rede, a regra do NC (cadeia mais longa vence) exige que o traidor invista mais trabalho computacional (PoW) do que todos os generais honestos combinados.

6.1. O Limite BFT do NC: O Ataque de 51%

Assim como o PBFT falha se os traidores controlarem $\geq 1/3$ dos nós, o Consenso Nakamoto também tem um limite BFT claro. O NC é seguro desde que os nós honestos controlem a maioria do poder computacional. O modelo de falha é, portanto, $f < 50\%$ do poder de hash.

O Ataque de 51% é simplesmente o nome dado à violação dessa suposição de segurança: Se um único general traidor (ou um cartel de traidores) conseguir controlar

mais da metade do poder de computação (taxa de hash) da rede, ele quebra o consenso. Com 51% do poder, o traidor pode garantir que sua cadeia maliciosa (contendo, por exemplo, um gasto duplo) crescerá mais rápido que a cadeia honesta. Eventualmente, a cadeia do traidor se tornará a cadeia mais longa, e todos os nós honestos, seguindo a regra do NC, a aceitarão como a verdade, revertendo transações legítimas.